



Android lecture 6

Release process, Best practices

2024

Lukáš Prokop

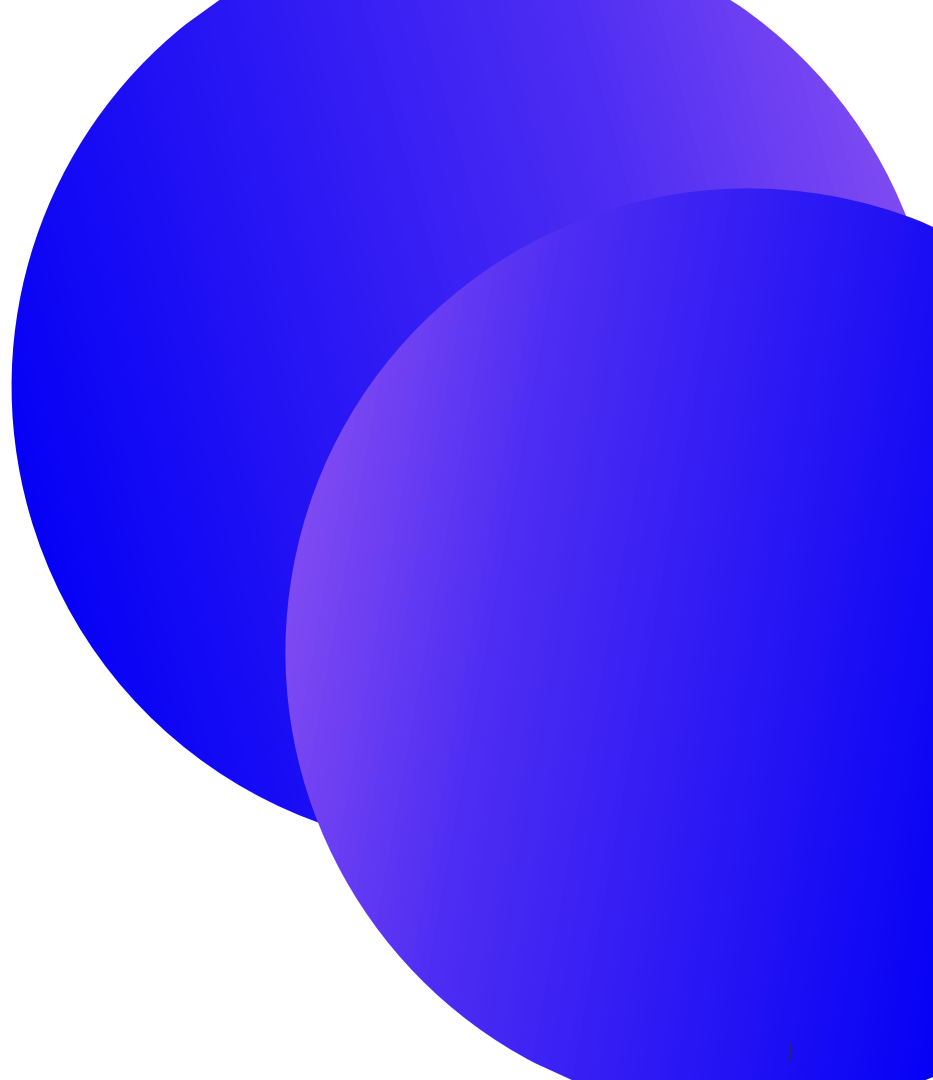
Simona Kurňavová



Agenda

- Release process
- Good practices in Android development

Application release



Gather materials for release

- **Cryptographic keys**
 - Apps in store are digitally signed, by developer certificate
 - Identify app developer
 - Self-signed certificate is enough
 - Needs to end after 22 October 2033 (>25 years recommended)
 - Protect your private key and password, it is not possible to update application if it is lost
- **End-user license agreement (EULA)**
 - User should know if you gather some data
 - Not required but recommended
 - GDPR

Gather materials for release

- **Application icon**
 - Visible in launcher, settings or other applications
 - High-res assets for google play store listing
- **Misc. materials**
 - Promo and marketing materials
 - Promotional text and graphic for store listing
- **Changelog**

Before release

- Delete unused parts (sources, resources, assets)
- Package name
 - <https://play.google.com/store/apps/details?id=com.avast.android.mobilesecurity>
- Turn off debug features
 - Delete Log.* calls
 - Remove android:debuggable flag from AndroidManifest
 - Remove Debug or Trace calls
 - If you using WebView ensure that debug is disabled, otherwise is possible to inject js code
- Clean up your directories, checks if libraries doesn't include some unnecessary files to your *.apk (*.proto, java manifests, ...)

Configure application for release

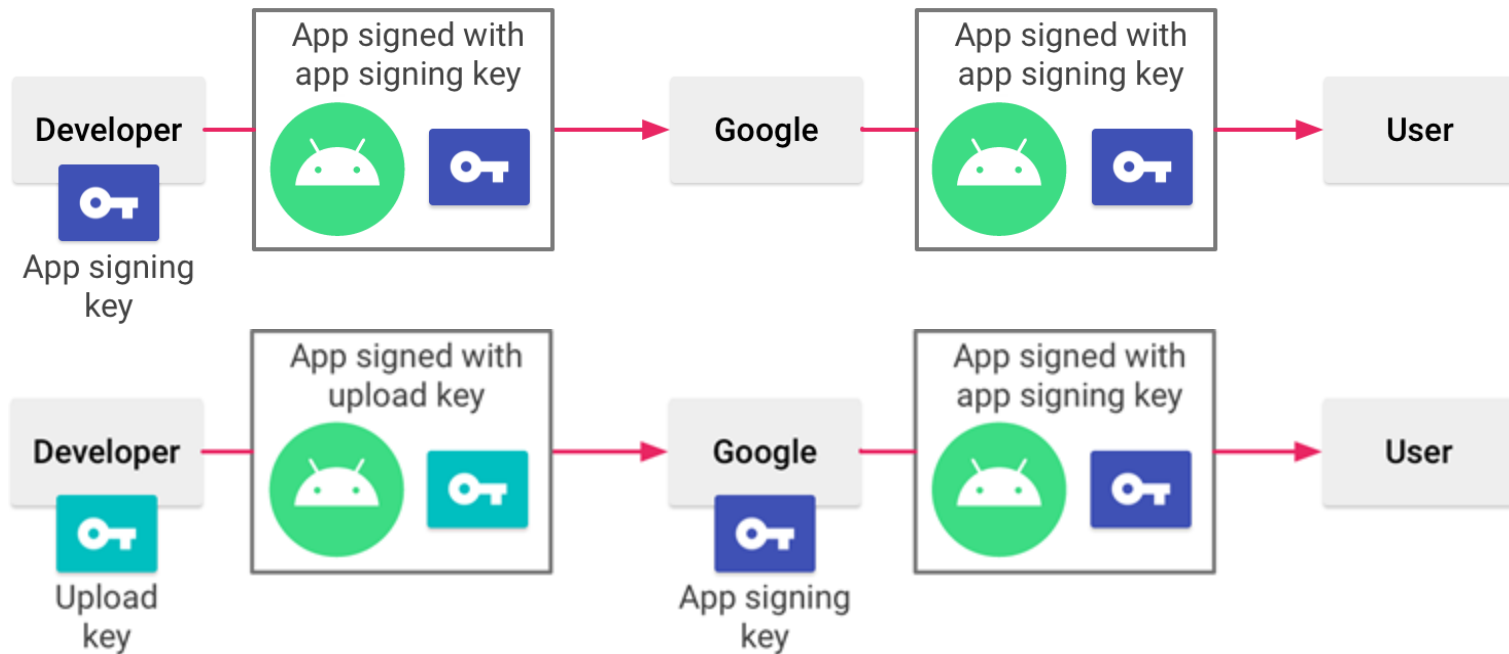
- **Review**
 - permissions - remove unnecessary
 - App icon and label
 - Version code and version name
 - Used URLs test vs. production backend
- **Check compatibility**
 - Support of multiple screens
 - Tablet mode

Build application for release

- **Signing**
 - Manually
 - Using Keytool and Jarsigner from JDK
 - Configure sign options in gradle
 - Android studio
 - Sign scheme v2 <https://source.android.com/security/apksigning/v2>
 - Sign scheme v3 <https://source.android.com/security/apksigning/v3>
 - Sign scheme v4 <https://source.android.com/security/apksigning/v4>
- **Obfuscation**
 - Use proguard to obfuscate your code, it is really easy to decompile application and find how it works, endpoints
- **Consider**
 - Who has access to your sign key
 - Signing server

Signing

- Sign vs upload key



Prepare external servers and resources

- Ensure that backend is running
- Check if app is switched to prod environment

Testing your application for release

- Regression test
- Test new features
- If it is possible test it on multiple devices, android versions
- Test multiple languages, including RTL
- Check lint for several issues

Publishing



- **Google play store**
 - Registration cost 25 USD
 - Reporting about installs
 - Crashes
 - If user sends it
 - Cloud test lab
 - Run monkey tests on multiple devices before releasing
 - Alpha/Beta groups
 - Distribution specific domain - internal apps
 - Staged rollout
 - API - import crash reports to bug tracker, upload new APK, ...

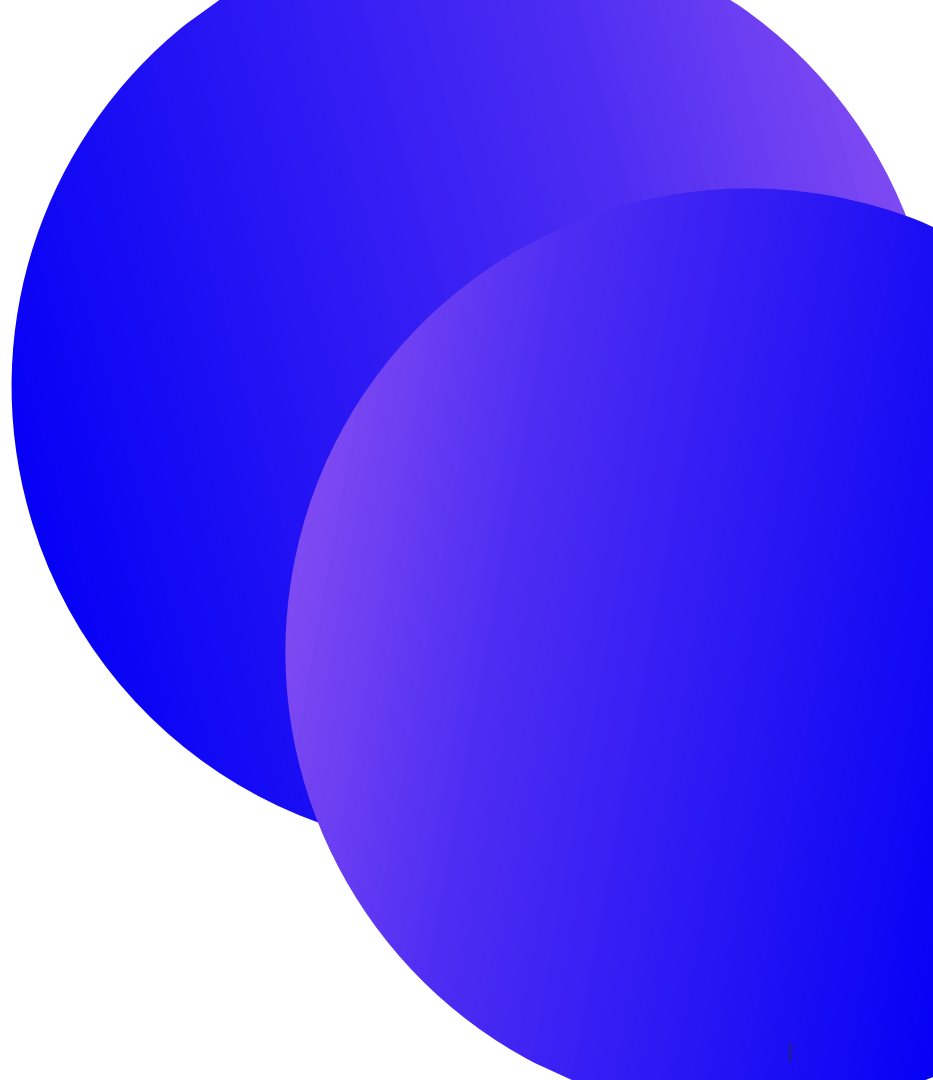
Publishing

- **Email, Web page**
 - Untrusted sources - security risk
 - Manual Updates
- **3rd party distribution**
 - Amazon
 - F-droid

After publish

- Monitor new crashes
- New permission slows down spreading between users on API < 23 (Marshmallow 6.0)
- Not good idea publish app before weekend or vacation

Best practices



SW development best practices

- Create libs for parts used in multiple projects (simplify maintenance, speed-up builds)
- Use git and gitflow
- Do code reviews
- Write tests (unit, UI)
- Ensure your code is readable, uniformly formatted and free of unused classes/methods/variables
- Comments for complex classes/methods/code.

Android best practices

SDK Configuration

- **minSdk**: Defines the minimum supported Android version. When in doubt, use the recommended version. Avoid going too low unless necessary, as it increases development complexity.
- **targetSdk**: Set this to the latest supported version to align with the latest platform features and security updates.
- **compileSdk**: Always use the latest available version for compilation.

Android best practices

Resources:

- *Strings*: Store all application texts (e.g., button labels, messages) in the `strings.xml` file. This practice aids in maintenance and facilitates providing translations if needed.
- *Images*: Prefer using vector images in XML format for scalability. If using bitmap images, provide multiple sizes (e.g., `hdpi`, `mdpi`, `xhdpi`, etc.) to support various screen densities.
- Other resources: colours, dimensions, etc. should be defined as resources.

Android best practices

Navigation

- Test navigation thoroughly to ensure all back buttons and navigation flows work as expected.
- Options for implementing navigation:
 - **Recommended:** Navigation Component, suitable for both Compose and Activities/Fragments.
 - For navigation between activities: use `startActivity(intent)`.
 - For navigation between fragments: `FragmentManager` to replace Fragments as needed (valid but error-prone; use of navigation component is preferred)

Android best practices

MVVM Architecture:

- Follow the **Model-View-ViewModel (MVVM)** pattern.
- Ensure that Activities, Fragments, and Composables contain no business logic. All business logic should reside in ViewModels (e.g. communication with database or API).

Android best practices

Persistence options:

- Simple primitive values: Data Store (preferred) or Shared preferences
- More complex data and lists: Database Room (preferred) or SQLite
- Backup and sharing across different devices/platforms: Server/Cloud services

Android best practices

Concurrency:

- All operations such as database access, reading/writing to files, preferences access, server communication, calculations, AI inference, etc. **shouldn't be done on Main/UI thread.**
- **Options:** Coroutines (recommended), Handler, Threads, Rxjava, etc.

Android best practices

Configuration Changes

- Ensure the app handles configuration changes (e.g. device rotation, light/dark theme switch or language changes)
- Preserve the screen's state during configuration changes using `ViewModel`, `rememberSaveable` in `Compose` or `onSaveInstanceState` in `Activity`
- Common issues to address:
 - Prevent losing user input in fields during config changes
 - Prevent reloading data during config changes

Thank you

Lukáš Prokop
Simona Kurňavová

Lukas.Prokop@gendigital.com

Simona.Kurnavova@gendigital.com

